Solution: Reverse Linked List (easy)

Problem Statement:

Given the `head` of a singly linked list, return the head of the reversed list.

Examples

Example 1:

- **Input**: `[3, 5, 2]`
- **Expected Output**: `[2, 5, 3]`
- **Justification**: Reversing the list `[3, 5, 2]` gives us `[2, 5, 3]`.

Example 2:

- **Input**: `[7]`
- **Expected Output**: `[7]`
- **Justification**: Since there is only one element in the list, the reversed list remains the same.

Example 3:

- **Input**: `[-1, 0, 1]`
- **Expected Output**: `[1, 0, -1]`
- **Justification**: The list is reversed, so the elements are in the order `[1, 0, -1]`.

Solution

To reverse a singly linked list, begin by setting up two pointers: 'previous' as null and 'current' as the head of the list. Traverse through the list, and during each step, reverse the current node's next pointer to point to the previous node. Then, update 'previous' to be the current node, and 'current' to its original next node. Continue this process until you reach the end of the list. When 'current' becomes null, the reversal is complete, and the 'previous' pointer will be at the new head of the list.

1. **Initialization**: Initialize three pointers: `prev` to `null`, `current` to the `head` of the list, and `next` to `null`.
2. **Traversal and Reversal**: Traverse the list. For each node, temporarily store the next node, update the current node's next pointer to point to the `prev` node, and move the `prev` and `current` pointers one step forward.
3. **Termination**: When the `current` pointer reaches the end of the list (`null`), the `prev` pointer will be pointing at the new head of the reversed list.

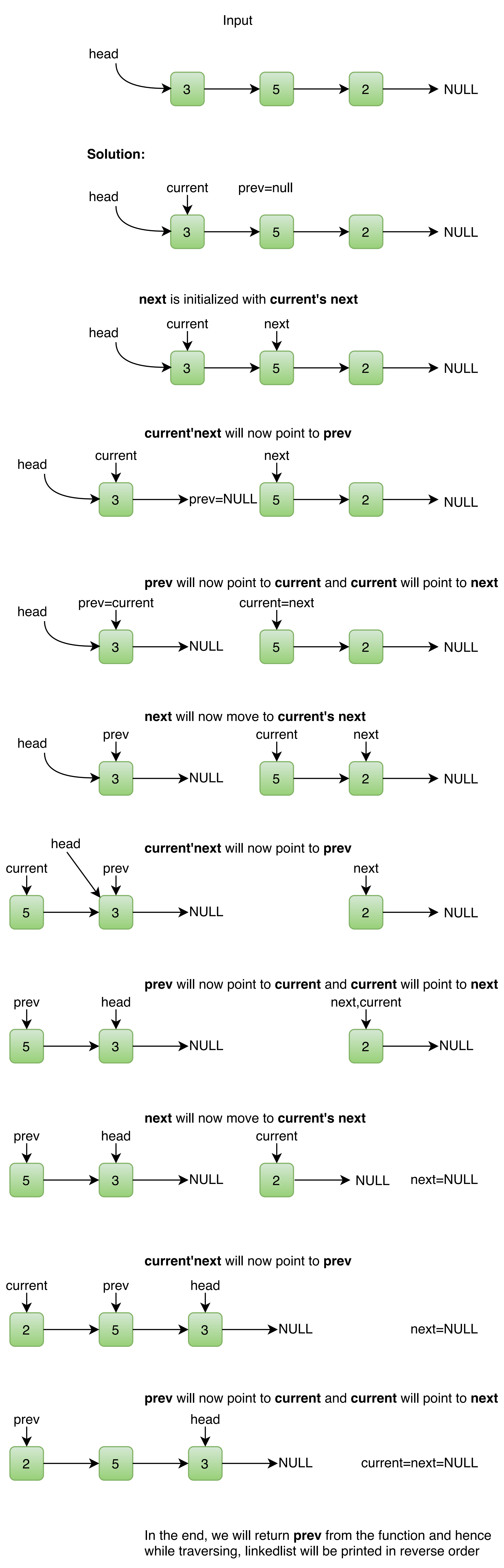
This algorithm will work because, during the traversal, we are reversing the direction of the pointers between the nodes, which effectively reverses the list. It is efficient and uses only a constant amount of extra space.

Algorithm Walkthrough:

Given the input list `[3, 5, 2]`:

- **Step 1**: Initialize `prev = null`, `current = 3`.
- **Step 2**: Traverse the list.
  - Store `next = 5`.
  - Update `current.next` to `prev`, so `3` now points to `null`.
  - Move `prev` to `3` and `current` to `5`.
- **Step 3**: Repeat Step 2.
  - Store `next = 2`.
  - Update `current.next` to `prev`, so `5` now points to `3`.
  - Move `prev` to `5` and `current` to `2`.
- **Step 4**: Repeat Step 2.
  - Since there is no next node, set `next = null`.
  - Update `current.next` to `prev`, so `2` now points to `5`.
  - Move `prev` to `2`, `current` becomes `null`, and we exit the loop.
- **Step 5**: The `prev` is now pointing to the head of the reversed list `[2, 5, 3]`.

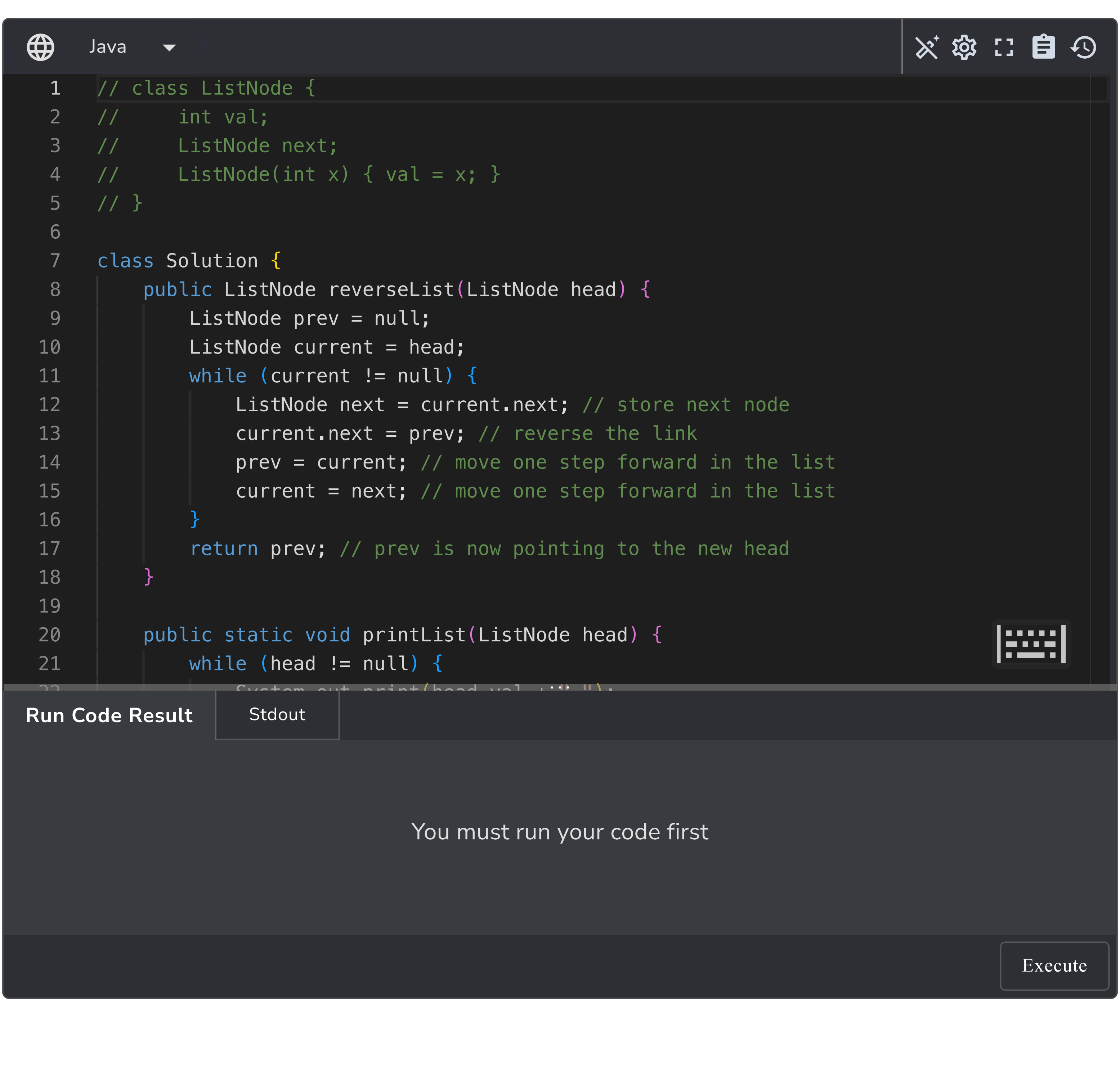
Here is the visual representation of the algorithm:


Input
head
3
5
2
NULL
Solution:
current
prev=null
next is initialized with current's next
next
current'next will now point to prev
prev=NULL
prev will now point to current and current will point to next
prev=current
current=next
next will now move to current's next
prev
next,current
next=NULL
current=next=NULL
In the end, we will return prev from the function and hence while traversing, linkedlist will be printed in reverse order

Reverse Linked List

Code

Here is the code for this algorithm:


Java
Run Code Result
Stdout
Execute

```java
// class ListNode {
//     int val;
//     ListNode next;
//     ListNode(int x) { val = x; }
// }

class Solution {
    public ListNode reverseList(ListNode head) {
        ListNode prev = null;
        ListNode current = head;
        while (current != null) {
            ListNode next = current.next; // store next node
            current.next = prev; // reverse the link
            prev = current; // move one step forward in the list
            current = next; // move one step forward in the list
        }
        return prev; // prev is now pointing to the new head
    }

    public static void printList(ListNode head) {
        while (head != null) {
```

You must run your code first

Complexity Analysis

Time Complexity

The time complexity of the reverse linked list algorithm is $O(n)$, where n is the number of nodes in the linked list. The algorithm traverses the entire linked list exactly once, performing a constant amount of work for each node (changing the next pointer and updating the current and previous pointers). Since the amount of work done is directly proportional to the number of nodes in the list, the time complexity is linear.

Space Complexity

The space complexity of the algorithm is $O(1)$, which represents constant space. This is because the algorithm only uses a fixed number of extra variables (previous, current, and next) to reverse the linked list, regardless of the size of the input linked list. No additional data structures or recursive stack space that scale with the input size are used, so the space used by the algorithm does not depend on the number of elements in the linked list.